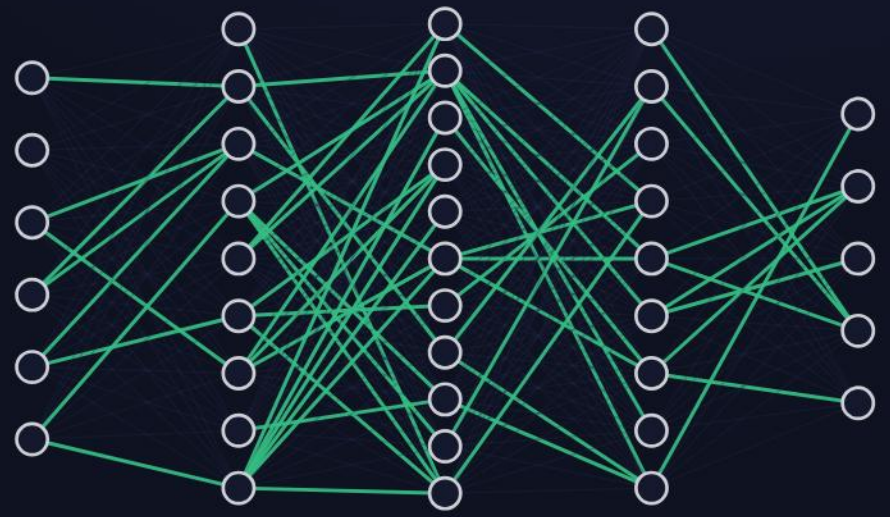
DEEP LEARNING Y SISTEMAS INTELIGENTES · CC3092 · PROYECTO FINAL

Lottery Ticket Hypothesis: ¿cuándo hace falta hacer rewind, y cómo se poda?



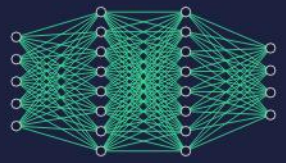
MLP, CNN, ResNet y ViT: dónde existe el boleto ganador, qué cambia con la complejidad, y por qué importa podar por capa o globalmente

José Antonio Mérida Castejón

2 de noviembre de 2026

Casi todos los pesos sobran: queda un boleto ganador

Red densa



Cada línea es un peso.

podar ~90%

Subred podada



Quedan los pesos de mayor |w|: ~10%.

Entrenada desde **los mismos valores iniciales**, esa subred aprende casi igual que la red completa: un **boleto ganador**.

Control: la misma estructura con pesos **nuevos**. Si rinde peor, importan los valores, no solo la forma.

1 · Entrenar
la red completa.

→

2 · Podar
20% de los pesos de menor magnitud.

→

3 · Regresar
los que quedan a un valor de referencia.

→

4 · Repetir
cada vez más pequeña (IMP).

primeras épocas — tiempo de entrenamiento →

θ_0: inicialización
ticket clásico

θ_k: época k
rewinding [2]

θ_T: final
red entrenada

El problema: en redes grandes, regresar a θ_0 deja de bastar con lr alto y hay que hacer rewind a θ_k [2]. ¿Qué cambia al subir la complejidad (MLP → CNN → ResNet → ViT), y qué decisiones de la poda cambian el resultado?

Cuatro hipótesis, y qué tan directamente se prueba cada una

H0

Existencia

En cada arquitectura hay una subred pequeña que iguala a la densa y le gana a la misma máscara con pesos nuevos.

prueba directa

H1

Complejidad → rewind

Mientras más compleja la red, más tarde en el entrenamiento aparece el ticket: hay que hacer rewind.

prueba parcial

1-2 puntos de rewind por arquitectura; sin barrido de *k*.

H2

Warmup ≈ rewinding

El warmup estabiliza el arranque, así que una red con warmup necesitaría menos rewind.

solo observacional

El warmup nunca se varió: está confundido con optimizador y datos.

H3

Alcance de la poda

a) En la CNN, global > por capa porque no vacía las capas pequeñas.
b) Las redes grandes necesitan poda global.

a) directa

b) no probada

Objetivo adicional. Mirar dentro de la red, no solo la accuracy: qué capas y qué pesos sobreviven (máscaras) y si la atención explica algo, en la CNN y en el ViT.

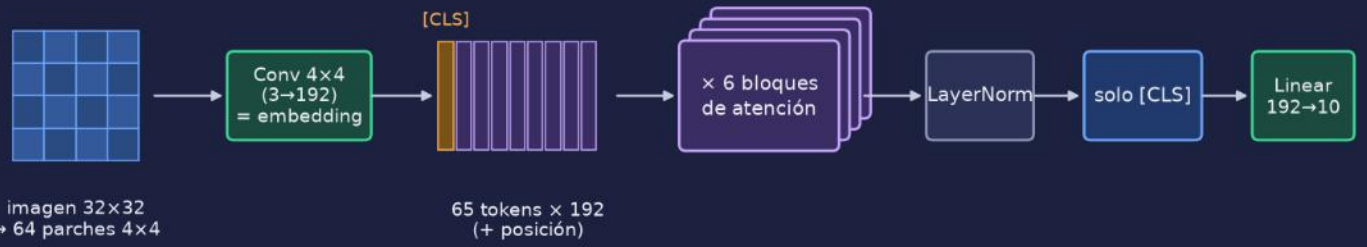
Enfoque. El **ViT** es donde más profundizamos (dos corridas e inspección completa); la **CNN**, donde se comparó poda por capa y global. El MLP y la ResNet son puntos de referencia.

Dónde más profundizamos: CNN y ViT

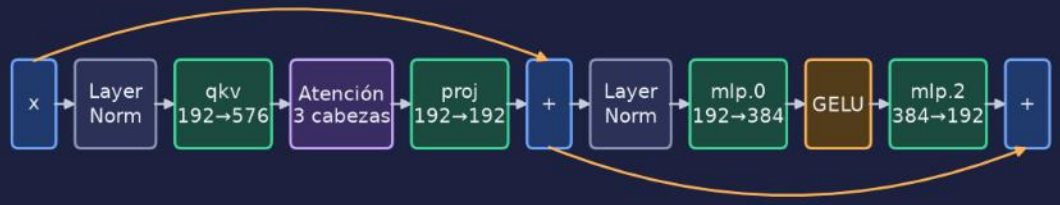
CNN · Conv-4 | ViT

clic para cambiar de red

ViT pequeño: 6 bloques, dimensión 192, 3 cabezas, ~1.8M pesos podables



Dentro de un bloque (se repite 6 veces)



arcos naranja: conexiones residuales (se suma la entrada)

verde: capa con pesos que se podan · morado: atención

ViT · CIFAR-10 · 1.8M pesos

Optimizador	AdamW 1e-3
04 · corto	20 ép. × 8 rondas · warmup 2 · rewind ép. 2
05 · largo	100 ép. × 8 rondas · warmup 5 · rewind ép. 5 · aumento fuerte
Poda	global (35% por ronda)
Qué prueba	H0, H1, H2 + inspección

Cada parche es un token; el [CLS] "mira" a los 64 parches (de ahí los mapas 8×8). 05 repite 04 porque su baseline (78.9%) parecía subentrenado.

Un mismo código de poda (`1th.py`) para las cuatro redes; el MLP (LeNet-300-100) y la ResNet-20 son puntos de referencia y sus diagramas están en el informe. Una semilla por experimento; 02b repite 02 cambiando **solo** el alcance de la poda.

Los modelos y `lth.py` son míos; la IA hizo checkpoints, gran parte de los experimentos y las figuras

Mi parte

- Tema, pregunta e hipótesis
- Los 4 modelos (`nn.Module`) y los loops de entrenamiento / evaluación
- `lth.py` : máscaras, IMP, rewinding, control aleatorio
- Correr y verificar los experimentos; análisis y conclusiones

Apoyo de la IA (Claude Code)

- Guardado de pesos y máscaras (checkpoints)
- Implementación de la mayoría de los cuadernos de experimentos (yo verifiqué lo central)
- Visualizaciones, diagramas y tablas del informe
- Revisión de código e hiperparámetros contra las fuentes
- Borradores de interpretación (contrastados con los datos)

Herramientas nuevas

marimo: notebooks reactivos guardados como `.py` , exportables a HTML.

uv: entornos reproducibles (`uv.lock`).

Claude Code: asistente en la terminal que lee el repo, edita y corre comandos con permiso.

1 · Yo
Escribí a mano los modelos, los loops de entrenamiento y `lth.py` .

→

2 · IA
Conectó `lth.py` al resto de los cuadernos, corrió los experimentos y agregó checkpoints y figuras.

→

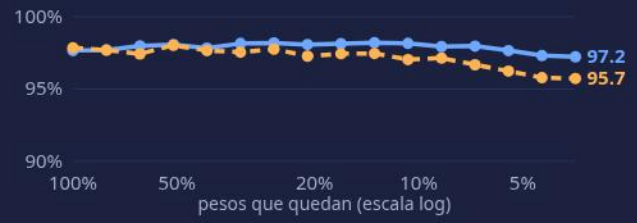
3 · Yo
Lo comparé con mi implementación y con los datos.

Dónde falló: agregó complejidad que no pedí, sacó la poda de `lth.py` , volvió a poner `print` que había quitado y presentó la atención como explicación. Cada número del informe sale de `results/*.json` o de los checkpoints; el historial está en §5.2.

Hay tickets en las 4 arquitecturas; la ResNet, con rewind

— Ticket (θ_0) — Ticket con rewinding (θ_k) --- Control: pesos nuevos Eje x: % de pesos que quedan (log); eje y: accuracy de prueba

MLP · MNIST



CNN · poda por capa



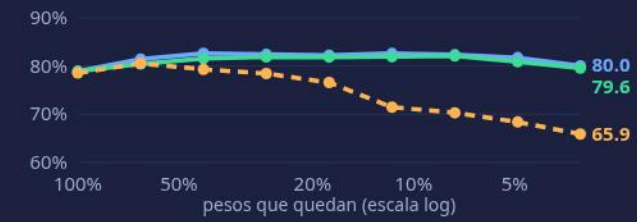
ResNet-20 · rewind época 1



CNN · poda global

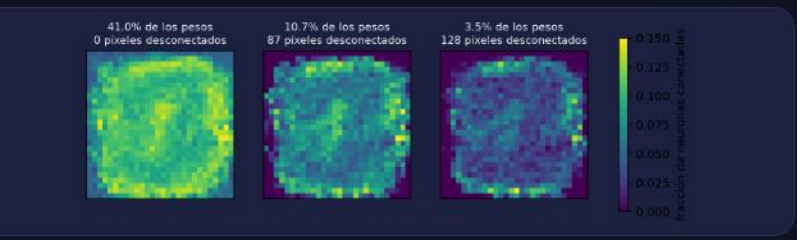


ViT · 20 épocas



ViT · 100 épocas





MLP, qué sobrevive: cada cuadro es la imagen de entrada de 28×28; oscuro = píxel casi sin conexiones. Con 3.5% de los pesos, los 128 píxeles del borde (siempre 0 en MNIST) quedan totalmente desconectados.

Densas: 97.7 · 73.0 / 74.0 · 90.9 · 78.9 · 86.0%. Ticket > control por 5-9 puntos (el ruido es ±0.6). ResNet: desde θ_0 = control (86.3 vs 86.5%); con rewinding, 89.6%.

Por capa, la CNN pierde detectores; global, no





Los 64 filtros de la primera convolución (cada cuadro = un filtro 3×3×3). Negro = posición podada · rojo = filtro sin ningún peso vivo

53.0% ticket, por capa

75.7% ticket, global

control: 61.2% / 69.4%

Filtro vacío: un detector (de bordes, colores) sin ningún peso vivo; su salida es constante, así que ese detector murió. Por capa: **7 de 64**, y **102 de 256** neuronas de fc2 ya no llegan a la salida. Global: **0**.

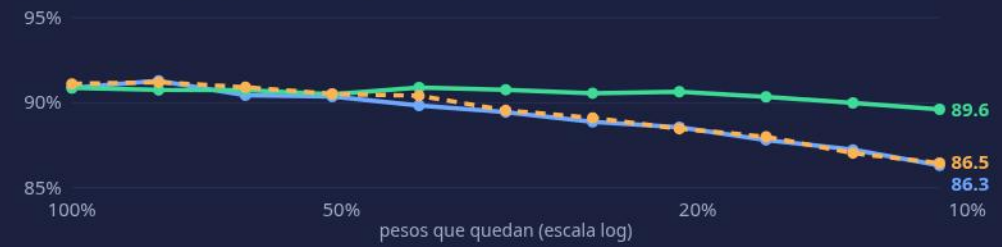
¿Por qué? La mediana de |w| es **0.105** en conv1 y **0.0086** en fc1 (12× menos): un umbral global tira primero los pesos diminutos de fc1 y casi no toca a conv1. La poda por capa les exige perder lo mismo.

¿Y las redes grandes? Consistente, *no probado*: ResNet y ViT nunca se podaron por capa.

La ResNet necesita rewind; el ViT, solo a poca densidad

— Ticket (θ_0) — Ticket con rewinding (θ_k) - - - Control: pesos nuevos

ResNet-20 · rewind a la época 1



ViT · 100 épocas · rewind a la época 5



	Rewind	¿Hizo falta?	Evidencia
MLP	–	no	θ_0 iguala a la densa hasta 3.5%
CNN	–	no	ídem (poda global)
ResNet	época 1	**sí**	desde θ_0 = control (86.3 vs 86.5%); con rewind 89.6%
ViT 20 ép.	época 2	no	rewind ≈ ticket (-0.7 pts)
ViT 100 ép.	época 5	**solo a poca densidad**	+3.4 pts a 7.5%, +3.3 a 4.9%, +1.2 a 3.2%

H1: parcial

MLP y CNN no necesitan rewind y la ResNet sí, pero el ViT, la más compleja, no lo necesita con densidades medias: la relación no es monótona. Solo hay uno o dos puntos de rewind por arquitectura.

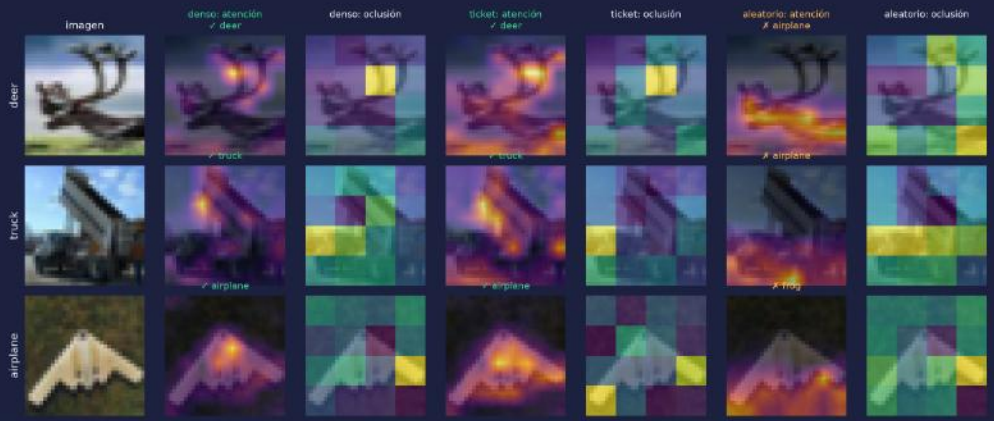
H2: compatible, no probada

La ResNet (sin warmup) necesita rewind y el ViT (con warmup) casi no, igual que en [1, 2]. Pero difieren también en optimizador, normalización y datos. Prueba directa pendiente: ResNet con warmup y ViT sin él.

El ViT largo parte de un baseline mejor (78.9% → 86.0%) y su ticket sigue por encima de la red densa hasta ~12% de los pesos, así que el resultado de 04 no era solo un baseline débil.

ViT: poda desigual, y la atención no explica





Capas. Con 3.2% de los pesos, `head` conserva 21% y `patch_embed` 7%, y `mlp.2` solo 1.3%.
Significa: la poda quita primero lo redundante (las capas MLP internas); la entrada y la salida son pequeñas y cada peso pesa más. Mismo patrón que la CNN; es una hipótesis.

Atención vs. oclusión. Los mapas del denso, ticket y rewinding se parecen entre sí (0.63–0.73) y los del aleatorio no (0.17–0.28), pero al tapar parches la correlación con la atención es **≈ 0** (0.05 denso, 0.06 ticket, 0.12 aleatorio; 150 imágenes).
Significa: la atención **no** dice qué usa el modelo; los mapas son descripción, no explicación. No se concluye por qué falla el control. (Imágenes de la figura elegidas.)

Clases. El ticket queda dentro del ruido (±2.5) en casi todas; el aleatorio pierde en todas (hasta -14.6, pájaro).
Significa: la estructura sola no basta; los valores iniciales cargan información.

Conclusiones

Hipótesis	Veredicto	Base
H0 existencia	**apoyada**	ticket ≥ denso y > control en las 4 arquitecturas (ResNet: con rewind)
H1 complejidad → rewind	**parcial**	ResNet sí; MLP y CNN no; ViT solo a poca densidad; sin barrido de k
H2 warmup ≈ rewind	**compatible, no probada**	patrón ResNet vs. ViT, confundido con optimizador y datos
H3a global > por capa (CNN)	**apoyada**	53.0% → 75.7%; sin filtros vacíos; pesos de capas pequeñas 12× mayores
H3b redes grandes → global	**consistente, no probada**	ResNet y ViT nunca se podaron por capa

Límites

Una semilla por experimento; ~10,000 imágenes de prueba (diferencias ≤ 1 punto son ruido); sin barrido de k; warmup sin variar; sin transferencia entre datasets.

Siguiente

ResNet con warmup y ViT sin warmup (H2); ViT y ResNet con poda por capa (H3b); barrido de k con varias semillas y McNemar.

Referencias. [1] Frankle & Carbin, *The Lottery Ticket Hypothesis*, ICLR 2019. [2] Frankle et al., *Stabilizing the LTH*, 2019. [3] Renda et al., *Rewinding and Fine-tuning*, ICLR 2020. [4] Touvron et al., *DeiT*, ICML 2021. [5] Shen et al., *Data Level LTH for ViTs*, IJCAI 2023. Repositorio: github.com/TonitoMC/lottery-ticket-hypothesis Video: ENLACE_YOUTUBE